

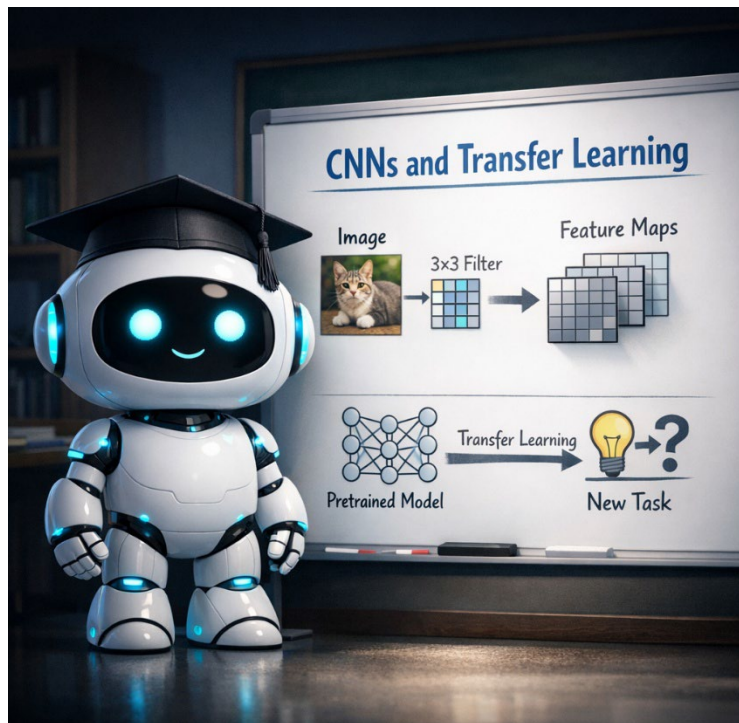
ITAI 1378

Computer Vision for Artificial Intelligence

Module 05

Convolutional Neural Networks and Transfer Learning

How vision became practical: networks that see locally, share filters, and reuse what others have learned.



uthor: Patricia McManus

Collaborator: Claude (Anthropic)

Welcome to Module 05

This is the module where everything you have learned so far becomes practical. Module 03 taught you how classical machine learning sees images. Module 04 taught you neural networks at a foundation level. Module 05 puts the two together into the architecture that powered the deep learning revolution in computer vision: the convolutional neural network.

You will leave this module able to do two things. First, you will understand how a CNN works from the inside out. You will know what a convolution actually is, why pooling matters, and how the famous architectures like LeNet, AlexNet, ResNet, and EfficientNet built on each other. Second, you will know how to take a CNN that someone else has already trained and adapt it to your own problem. This second skill, transfer learning, is the single most important practical technique in modern computer vision. Almost nobody trains a CNN from scratch in industry anymore. They start with a pretrained model and fine tune it. By the end of this module, that will be you too.

Like all booklets in this course, this is an evergreen reference text. There are no due dates, no Canvas screenshots, no semester specific notes. It is the kind of document you keep on your desk while you work, or share with a colleague who needs to come up to speed quickly. Code examples and frameworks will evolve. The concepts in this booklet, the why behind how a convolution works, will not.

KEY TAKEAWAY | What this module will give you

You will be able to explain what a CNN does at every layer, identify which classic architecture fits your problem, take a pretrained model from a library like torchvision or Hugging Face and fine tune it on your own data in an afternoon, and avoid the four or five common pitfalls that trip up beginners. That is the workforce baseline for a CV engineer in 2026.

Part 01 | Why Convolutional Neural Networks Exist

Before we talk about how a CNN works, you should know why it exists. The CNN was invented to solve a specific problem that earlier neural networks could not handle: images. This part traces the path from the basic neural network you learned about in Module 04 to the convolutional architecture that took over the field. Understanding this path will give you the intuition you need for everything that follows.

The Problem: Basic Neural Networks Drown in Pixels

In Module 04 you learned about the multi layer perceptron, or MLP. It is the simplest kind of neural network: a stack of layers, each layer fully connected to the next. When you give it a list of numbers, like the price of a house or the readings from a sensor, it produces good results. The problem comes when you try to feed it an image.

A modest color image, say 224 pixels wide and 224 pixels tall, contains over 150,000 numbers (one for red, green, and blue at every pixel). To feed it into an MLP, you flatten the whole image into one long list of 150,000 values. Then every neuron in the first hidden layer connects to every one of those 150,000 inputs. If your first hidden layer has just 1,000 neurons, that is 150 million weights in the first layer alone. The network becomes enormous, slow to train, and prone to overfitting on small datasets.

Worse than the size problem is the structure problem. An MLP treats every input pixel as if it were independent of every other pixel. It has no idea that pixel (10, 10) and pixel (10, 11) are right next to each other. It has no built in sense that a cat in the top left of the image is the same kind of pattern as a cat in the bottom right. The MLP has to learn from scratch, with brute force parameter count, what should be obvious from the start.

MISCONCEPTION ALERT | Bigger is not better for images

It is tempting to think that adding more layers or more neurons will fix the MLP problem. It will not. The MLP architecture is fundamentally mismatched to image data. The fix is not more capacity. The fix is a different architecture that takes spatial structure seriously by design. That is the CNN.

The CNN Answer: Three Big Ideas

Convolutional neural networks fix the MLP problems by changing the architecture. Three ideas do most of the work. Master these three and the rest of the module is straightforward.

Idea 1: Look at Small Patches, Not the Whole Image

Instead of connecting every neuron to every pixel, a CNN neuron looks at a small neighborhood of pixels, like a 3 by 3 or 5 by 5 patch. This dramatically reduces the number of weights. A neuron looking at a 3 by 3 patch needs only 9 weights, not 150,000. And it makes biological sense too: real visual neurons in animal brains also have small receptive fields.

Idea 2: Share the Same Filter Everywhere

The same small filter slides across the whole image and is applied at every position. The weights are shared. If the filter detects vertical edges in one part of the image, it detects vertical edges everywhere else too. This solves the location problem from the MLP: a cat in the top left and a cat in the bottom right activate the same filters. The network gets translation invariance for free, just from the architecture.

Idea 3: Stack Layers to Build a Hierarchy

CNNs stack layers on top of each other. Early layers learn to detect simple patterns like edges and color blobs. Middle layers combine those into shapes and textures. Deep layers combine those into whole objects like faces and cars. The hierarchy is not designed by humans. It emerges from training. This is one of the most beautiful properties of deep learning, and it is what makes CNNs so powerful on images.

DEEPER LOOK | Receptive fields in plain English

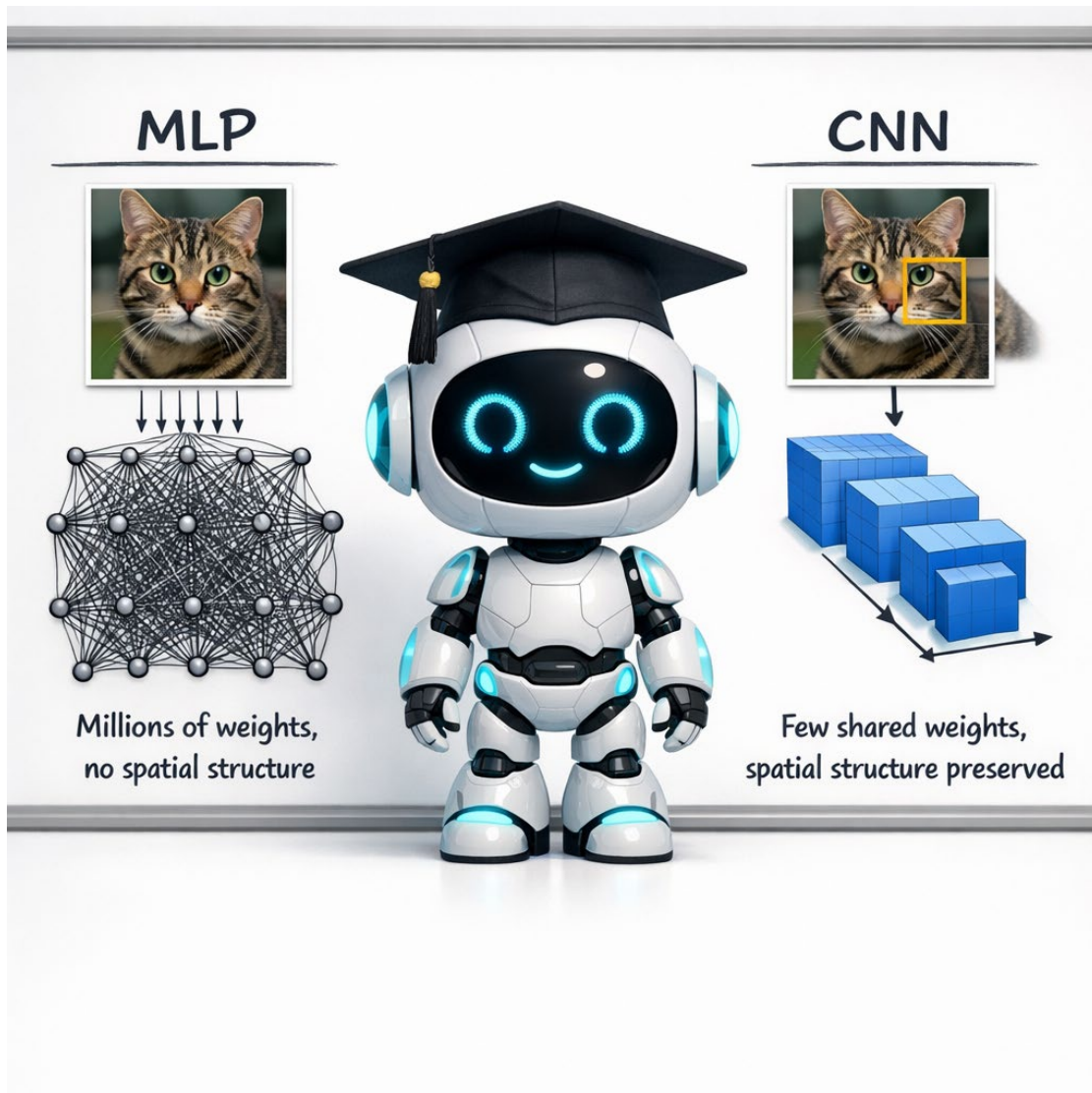
Think of a receptive field as the patch of the original image that a neuron can see. A neuron in the first layer of a CNN sees only its small filter neighborhood, maybe 3 by 3 pixels. But a neuron three layers deep gets its inputs from many first layer neurons, so indirectly it sees a much bigger chunk of the image. The deeper you go, the larger the receptive field grows. That growth is exactly what lets deep layers detect whole objects when early layers can only detect edges. No equations needed; just remember: deeper neurons see more.

Translation Invariance Without the Buzzwords

You will see the phrase translation invariance over and over in CV. It just means that a pattern is recognized the same way no matter where in the image it appears. A face is a face whether it is centered, in the corner, or partly off the edge. CNNs handle this naturally because of weight sharing: the same filters scan every position, so the same response fires wherever the pattern is. MLPs do not have this property. They have to learn the same pattern over and over for every position, which wastes both data and parameters.

INDUSTRY PRACTICE | Why CNNs are still the workforce default in 2026

Vision transformers (M08) get the headlines, but CNNs still run most production computer vision systems in 2026. Why? They are smaller, faster, work well with less data, deploy easily to edge devices, and have a mature tooling ecosystem (torchvision, timm, OpenCV interop). For 95 percent of real CV jobs in industry, the first thing you reach for is a pretrained CNN backbone. Transformers come in when the dataset is huge and the budget is generous.



KEY TAKEAWAY

Basic neural networks fail on images because they have too many weights and no built in sense of where pixels are relative to each other. CNNs fix both problems by looking at small local patches with shared filters, and by stacking layers to build a hierarchy of features. Three ideas: local patches, shared filters, hierarchical layers.

☒ **KNOWLEDGE CHECK** | Quick check on motivation

1. Roughly how many weights does an MLP need in just the first layer to process a 224 by 224 color image into a hidden layer of 1,000 neurons?
2. What does translation invariance mean? Why do CNNs have it naturally but MLPs do not?
3. Name the three big architectural ideas behind CNNs.

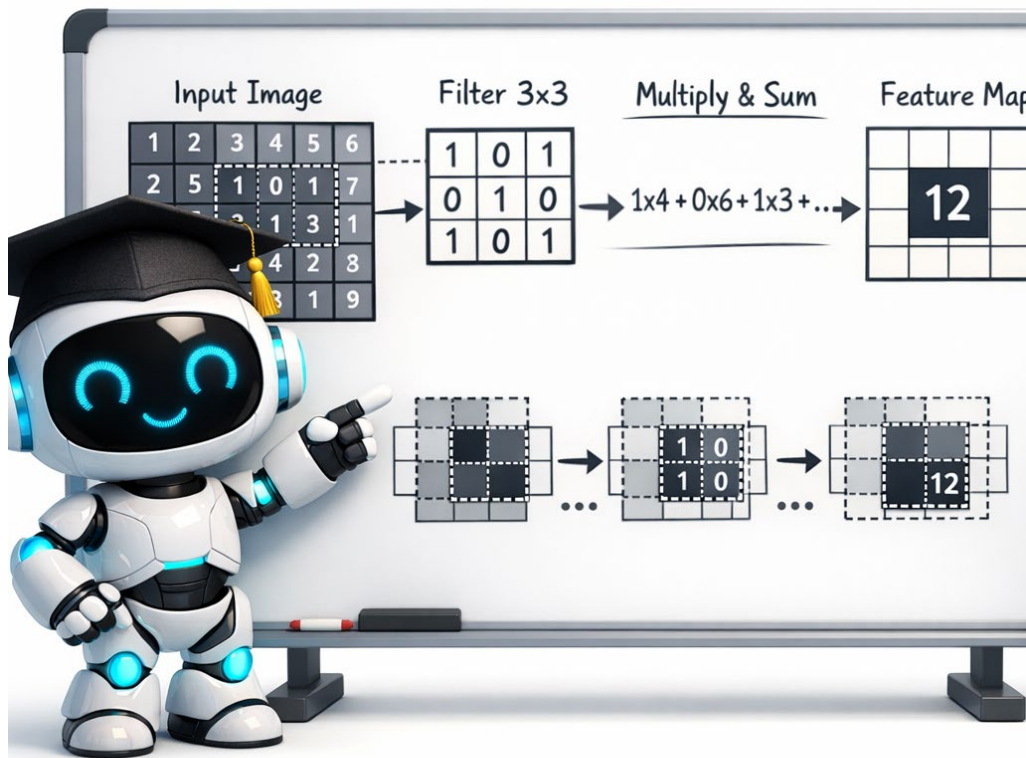
Part 02 | How a Convolution Actually Works

The word convolution sounds intimidating. The operation it describes is not. This part walks through what a convolution is doing in plain English, then introduces the small set of vocabulary words you need to read CNN code or talk about CNN architectures in a job interview.

The Filter and the Sliding Window

Start with a tiny piece of code, a filter. A filter is a small matrix of numbers, usually 3 by 3, sometimes 5 by 5 or 7 by 7. The filter is the pattern detector. The convolution operation slides this filter across the input image, position by position. At each position, the filter overlaps with a small patch of the image. The values in the filter get multiplied with the corresponding pixel values in the patch, all the products are added together, and the result is written into the output at that position.

Imagine you have a rubber stamp and a sheet of paper. The stamp is the filter. The paper is the image. You press the stamp at the top left corner and write down what it says. You move it one position to the right and press again, writing down the new result. You keep going across the whole sheet. When you are done, you have a new piece of paper, the same size or smaller than the original, where each spot tells you how strongly the stamp pattern was found at that position in the original image. That new piece of paper is called a feature map.



Feature Maps: What Comes Out

The output of a convolution is called a feature map. Where the input was an image, the feature map is a new picture that highlights wherever the filter pattern was found. If the filter detects vertical edges, the feature map will be bright wherever there is a vertical edge in the input and dark everywhere else. A CNN layer does not produce just one feature map; it produces many, one for each filter the layer has. A typical first layer might have 32 or 64 different filters, each looking for a different basic pattern.

** MATH MOMENT (optional) | What is in a filter, really**

A 3 by 3 filter is just nine numbers arranged in a grid. Multiply each number with the matching pixel under it, add the nine products, write the result. That is one filter application. Do this at every position in the image, and you have applied the filter to the whole image. The result is the feature map. The framework does the arithmetic for you; you never type out the multiplications. What matters is understanding that the filter values are the learned pattern, and the feature map shows where in the image that pattern appears.

Hand Designed Filters Versus Learned Filters

Here is the difference between Module 03 and Module 05 in one sentence. In classical computer vision, you designed the filters. You chose values that would detect edges, or smooth noise, or sharpen detail. In a CNN, the filter values are not chosen by humans. They are learned automatically from the training data through gradient descent. The network discovers what filters it needs for the problem at hand.

This shift is enormous. In M03 you needed an expert to design good features for each new problem. In a CNN, you give the network enough labeled examples and it discovers its own features. The cost is that you need a lot of labeled data and a lot of compute. The benefit is that you get features tailored to your specific problem, often better than anything a human would design by hand.

** THINK ABOUT IT | What does the network actually learn?**

If you visualize the filters that a trained CNN has learned (and there are tools that let you do this), the first layer almost always looks the same regardless of the task: oriented edge detectors, color blob detectors, simple gradients. The biology echoes are striking. Animal visual cortex starts with similar detectors. This is one of the deepest results in deep learning: the early layers learn universal visual features that generalize across problems. That is the foundation that transfer learning rests on.

Stride and Padding

Two small details control the shape of the output feature map: stride and padding. They are the parameters you will set every time you create a convolutional layer in code, so it is worth understanding them properly.

Stride: How Far the Filter Moves Each Step

Stride is the number of pixels the filter moves between positions. The default is stride one, which means the filter shifts one pixel at a time. The output feature map is roughly the same spatial size as the input. With stride two, the filter skips every other position, and the output is half the spatial size of the input. Bigger stride means smaller output and faster computation, but it skips information.

Padding: What Happens at the Edges

Without padding, the filter cannot fully overlap with pixels near the edge of the image, so the output feature map ends up slightly smaller than the input. Padding adds a border of zero pixels around the image to fix this. Same padding adds enough border so that the output stays the same size as the input. Valid padding adds no border and lets the output shrink. Both are common; same padding is the default in most modern architectures.

INDUSTRY PRACTICE | Reading PyTorch code

When you see `nn.Conv2d(in_channels=3, out_channels=64, kernel_size=3, stride=1, padding=1)` in a PyTorch model, you are looking at one convolutional layer. It takes 3 input channels (RGB), produces 64 output feature maps, uses 3 by 3 filters, slides them one pixel at a time, and pads to keep the output size the same. Every CNN you read will have lines that look like this. Get fluent with reading them; they tell you exactly what the network is doing.

KEY TAKEAWAY

A convolution is a small filter that slides across the image, producing a feature map that highlights where the filter pattern was found. The filter values are learned from data, not designed by humans. Stride controls how far the filter moves; padding controls what happens at the edges. These four words, convolution, filter, feature map, stride, are the vocabulary for the rest of CNN literature.

KNOWLEDGE CHECK | Quick check on convolutions

1. Describe in plain English what happens when a 3 by 3 filter is applied to a single position in an image.
2. What does a feature map represent?
3. If you use stride 2 with no padding on a 28 by 28 input, will the output be larger, smaller, or the same size? Roughly what size?

Part 03 | Building a Full CNN

A single convolution is just one ingredient. A working CNN needs several different kinds of layers, stacked together in a specific pattern. This part introduces the rest of the ingredients (pooling, activation functions, fully connected layers) and shows the standard CNN recipe that every modern architecture follows.

Pooling: Shrinking the Feature Map

After a few convolutional layers, you usually want to make the feature maps smaller. Smaller feature maps mean fewer numbers to process, faster training and inference, and more robustness to small shifts in the input. The standard tool for this is pooling. Pooling slides a small window across the feature map (a 2 by 2 window is common) and replaces the values in that window with a single summary value.

Max Pooling Versus Average Pooling

Two common pooling operations exist. Max pooling takes the biggest value in the window, and it is the default in modern CNNs. The idea is that you only care whether the pattern was detected somewhere in the window, not exactly where. Average pooling takes the mean of the values in the window. It is gentler but loses more information. In practice, almost every modern CNN uses max pooling between convolutional blocks.

MISCONCEPTION ALERT | Pooling is not just for size

Some students think pooling exists only to shrink the feature map for efficiency. That is half of the story. The other half is that pooling adds robustness to small shifts. If a pattern appears slightly to the left or to the right in the input, max pooling will produce the same output. This baked in invariance is part of why CNNs generalize well from limited data.

Activation Functions in a CNN Context

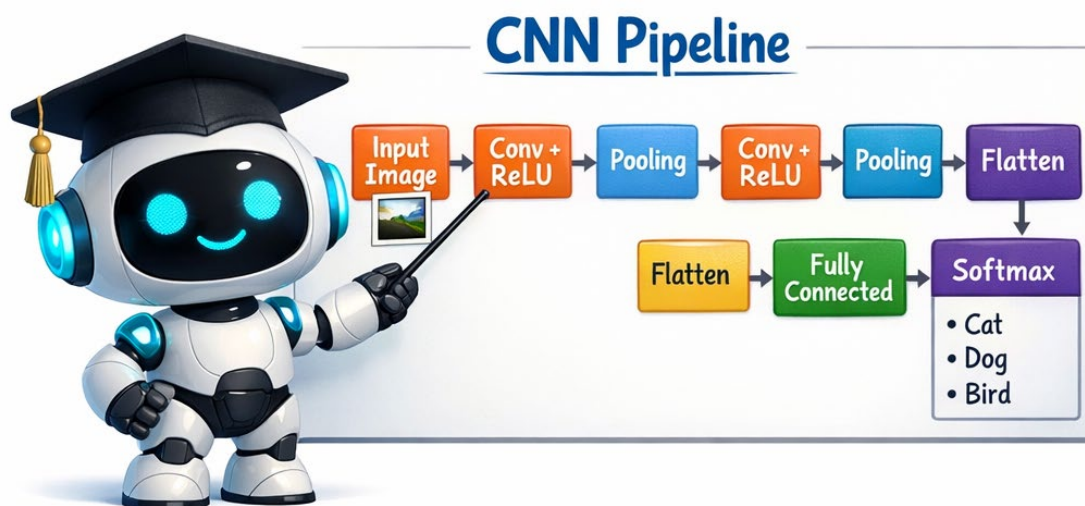
Between every convolutional layer and the next, the feature maps pass through an activation function. You learned about activations in Module 04: their job is to introduce non linearity so the network can learn complex patterns. In a CNN, the activation function is almost always ReLU, the rectified linear unit. ReLU keeps positive values and zeroes out negative ones. It is fast to compute, plays well with gradient descent, and has been the workhorse since AlexNet introduced it in 2012.

You may occasionally see variants like Leaky ReLU, GELU, or Swish in modern architectures. They are minor tweaks to address specific training issues. For your first CNN, ReLU is the safe choice and what every tutorial assumes.

Fully Connected Layers: The Classifier on Top

After several rounds of convolution, activation, and pooling, the network has converted the input image into a small stack of high level feature maps. The final step is to use those features to make a decision: what class is this image? For that, the feature maps are flattened into a single long vector, and then one or two fully connected layers (the kind you learned about in Module 04) make the final classification.

This is the part of the network that looks like an MLP. The reason it is small now is that all the heavy lifting was done by the convolutional layers, which reduced the image to a compact set of features. By the time the fully connected layers see the data, they only need to combine a few hundred or few thousand numbers into a class prediction. The full connectivity is fine at this point because the input is no longer high dimensional pixel data.



The Full CNN Recipe in One Sentence

Every modern CNN follows the same template: alternate convolutional blocks with pooling, stack several of them, flatten, finish with a fully connected classifier. Different architectures vary in how many layers, what filter sizes, where the pooling goes, and what extra tricks are added (like residual connections or attention). But the underlying recipe stays the same across decades of CNN research.

INDUSTRY PRACTICE | Where to start when designing a CNN

Senior CV engineers rarely design CNNs from scratch in 2026. They start with a published architecture that has already been validated, like ResNet50 or EfficientNet B0, and modify only what they need to. The recipe in this part is for understanding, not for original design. Once you understand it, you can read someone else's architecture and customize it for your problem. That is the workflow.

The Hierarchy That Emerges From Training

One of the most beautiful facts about CNNs is that the hierarchy of features is not designed by humans. Once you stack the layers and start training, the network discovers what each layer should detect on its own. If you visualize what a trained CNN has learned, the pattern is remarkably consistent. Early layers detect oriented edges and color blobs. Middle layers detect textures, simple shapes, and parts of objects (an eye, a wheel, a leaf vein). Late layers detect whole objects (a face, a car, a cat).

This emergent hierarchy is what makes CNNs interpretable in ways that other deep learning approaches sometimes are not. You can look at the filters of a trained network and see what it has chosen to look at. You can visualize the feature maps to understand what the network is paying attention to. The hierarchy is also what makes transfer learning (Part 05) work: the early universal features can be reused for problems the network was never originally trained on.

CAREER CONNECTION | Vocabulary recruiters listen for

When asked 'walk me through a CNN' in an interview, weak candidates jump straight to 'there are convolutions and then pooling'. Strong candidates frame it conceptually: 'A CNN preserves spatial structure by applying local shared filters, builds a hierarchy of features through stacking, downsamples with pooling, and classifies with a small fully connected head at the end.' Then they can drill into details if asked. Vocabulary and structure first, mechanics second.

 KEY TAKEAWAY

A full CNN is a stack of convolutional blocks (convolution + activation + pooling) followed by a flatten step and a small fully connected classifier. ReLU is the default activation. Max pooling is the default downsampling. The hierarchy of features emerges automatically from training. Master this recipe and every CNN architecture from LeNet to EfficientNet is a variation on the same theme.

 KNOWLEDGE CHECK | Quick check on full CNN structure

1. Name the four kinds of layers (or operations) that appear in a typical CNN.
2. Why does max pooling provide robustness to small shifts in the input?
3. What does the flatten step do, and where in the network does it appear?

Part 04 | The Architecture Family Tree

Once you understand the basic CNN recipe, the next step is learning which specific architectures are out there and what each one contributed. This part covers five architectures that define the family tree: LeNet, AlexNet, VGG, ResNet, and EfficientNet. You will see these names every day in CV work, and recognizing them is part of the workforce baseline.

LeNet (1998): The First Practical CNN

LeNet was built by Yann LeCun and his team at Bell Labs. It was used to read handwritten digits on bank checks, and it ran in production for years. By modern standards it is tiny: just five layers, designed for 32 by 32 grayscale digit images. But the pattern it established (conv, pool, conv, pool, flatten, fully connected, classify) is exactly the recipe modern CNNs still follow today. LeNet showed that feature extraction and classification could be learned together, end to end, by a single network. That was a radical idea in 1998.

AlexNet (2012): The Spark

AlexNet, built by Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton at the University of Toronto, is the network that started the modern deep learning era. It won the 2012 ImageNet competition by a margin so large (over ten percentage points better than the second place finisher) that the entire computer vision research community pivoted to deep learning within two years. AlexNet was eight layers deep, much bigger than LeNet, and was trained on the ImageNet dataset of 1.2 million labeled images across 1000 classes.

What made AlexNet work were several innovations that have since become standard: ReLU activations instead of the old sigmoid (much faster training), GPU training (made the larger network feasible), dropout regularization (prevented overfitting on the still relatively small dataset), and data augmentation (artificially expanded the training set). Every modern CNN inherits these tricks.

CAREER CONNECTION | The AlexNet moment in the field

If you ever wonder why CV practitioners are obsessed with AlexNet, this is why. Before AlexNet, classical computer vision (Module 03 territory: HOG, SIFT, SVMs) was the entire field. After AlexNet, almost every research lab and production team switched to deep learning within two years. It is the single most important paper in modern computer vision. Any CV interview that touches on history will mention it.

VGG (2014): Going Deeper

VGG, from Oxford's Visual Geometry Group, asked a simple question: what happens if we just make the network deeper? The answer was that depth helps a lot, as long as you keep the architecture clean and simple. VGG networks came in two main flavors, VGG16 (sixteen layers) and VGG19 (nineteen layers). They used only 3 by 3 convolutions, with pooling between groups, and a couple of fully connected layers at the end. The simplicity made VGG easy to understand and modify, and the network is still used as a baseline in research papers today.

ResNet (2015): The Single Most Influential Idea

ResNet, from Microsoft Research, introduced a small architectural trick called the residual connection or skip connection that turned out to be transformative. The trick is this: every few layers, the input to the block is added directly to the output of the block, skipping over the layers in between. This sounds minor, but it changed everything. Before ResNet, networks deeper than about thirty layers became impossible to train because the gradients vanished as they were backpropagated through so many layers. With skip connections, networks could go to 50, 101, even 152 layers and still train successfully.

Today, ResNet50 (the fifty layer version) is probably the most widely used CNN backbone in production. Almost every transfer learning tutorial starts with it. Almost every published CV paper compares against it. The skip connection idea also influenced everything that came after, including the modern transformer architectures (M08).

INDUSTRY PRACTICE | ResNet50 is the workhorse

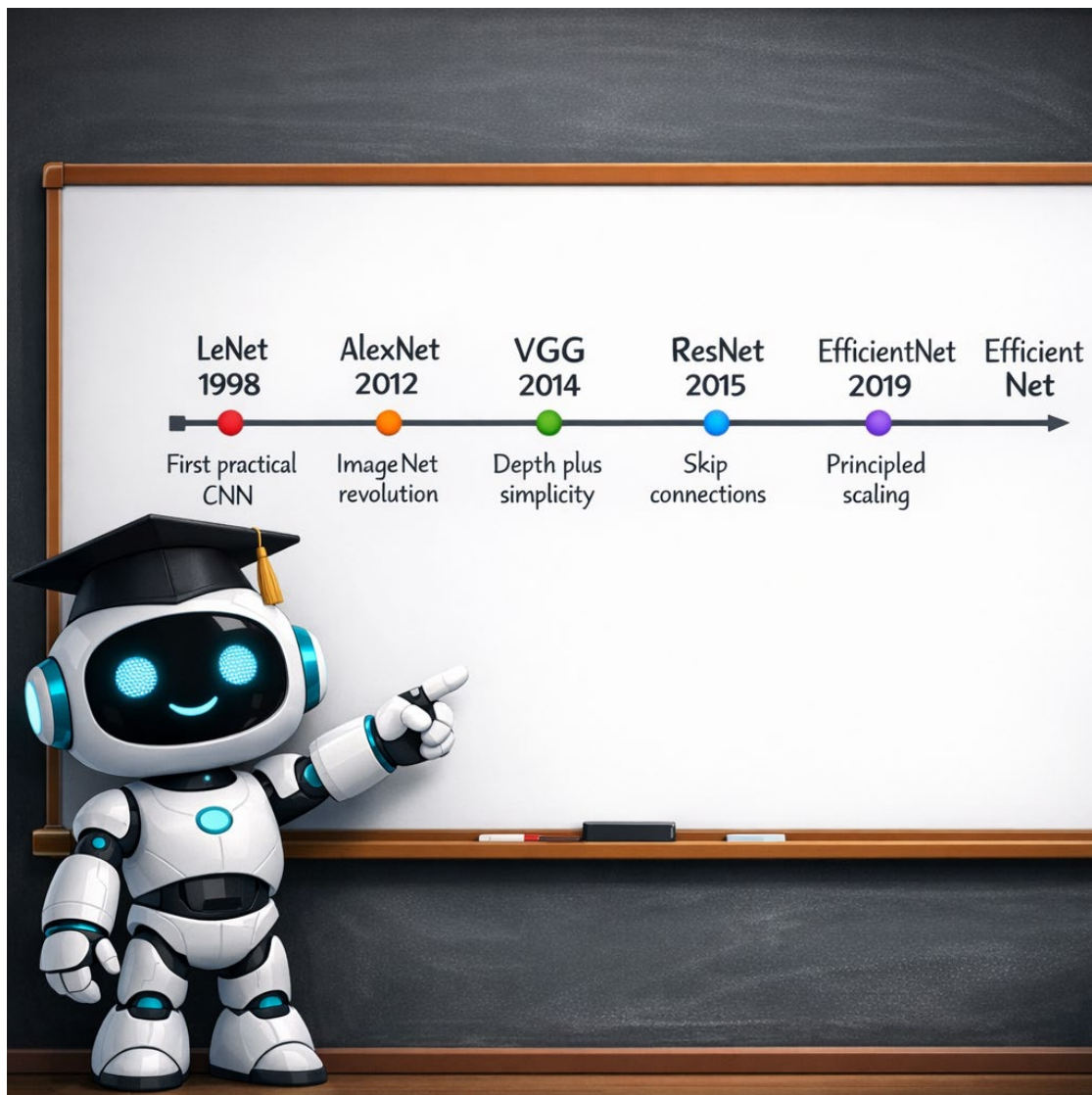
When a senior CV engineer says 'let's start with a baseline,' they almost always mean ResNet50 pretrained on ImageNet. The combination of decent accuracy, fast inference, well understood behavior, and ten years of community tuning makes it the universal starting point. Even when fancier architectures exist, teams default to ResNet50 first because everyone knows how to debug it. Familiarity wins over novelty in production.

EfficientNet (2019): Scale It Right

EfficientNet, from Google Brain, asked a more sophisticated question: when you scale a network up, what should you scale? Earlier work just added more layers (making it deeper) or more filters per layer (making it wider) or larger input images (making it higher resolution). EfficientNet showed that scaling depth, width, and resolution together using a principled rule gives much better accuracy than scaling any one

dimension alone. The result was a family of networks (EfficientNet B0 through B7) that achieved state of the art accuracy with far fewer parameters than competing architectures.

EfficientNet is the standard recommendation today when you need a smaller, more efficient model than ResNet50, especially for mobile or edge deployment. EfficientNetB0 in particular hits a sweet spot of accuracy and size that makes it widely used.



How They Connect: The Family Tree

The architectures in this part are not a random list. Each one solved a problem the previous one ran into. LeNet established the basic recipe but only worked at tiny scale. AlexNet showed that scaling the recipe up worked, given GPUs and a large dataset. VGG showed depth was a major win. ResNet showed how to

keep going deeper without breaking training. EfficientNet showed how to scale all three dimensions in proportion. Each architecture stands on the shoulders of the previous one.

In 2026, the cutting edge has moved beyond these classic CNNs to vision transformers and hybrid architectures (covered in M08). But the classic CNN backbones still dominate production because they are simpler, faster, smaller, and well understood. Knowing this family tree is part of the working CV vocabulary.

THINK ABOUT IT | Which architecture should you pick?

Faced with a new CV problem in 2026, how do you choose? Here is a working rule. If you have lots of data and lots of compute, try a vision transformer (you will learn about these in M08). If you need a strong, well behaved baseline that just works, use ResNet50. If you need a smaller model for edge deployment, use EfficientNetB0 or EfficientNetB2. If you are doing research and need maximum simplicity, use VGG. LeNet is purely historical at this point, but it is still the right teaching example.

KEY TAKEAWAY

Five architectures define the classic CNN family tree: LeNet started it in 1998, AlexNet scaled it up in 2012, VGG went deeper in 2014, ResNet made depth trainable with skip connections in 2015, and EfficientNet scaled smartly in 2019. ResNet50 is the production workhorse. EfficientNet is the efficient cousin. Knowing these names and what each contributed is the workforce vocabulary.

KNOWLEDGE CHECK | Quick check on architectures

1. Why did AlexNet matter so much to the field of computer vision?
2. What architectural innovation did ResNet introduce, and what problem did it solve?
3. If you need a CNN for mobile or edge deployment in 2026, which architecture family would you reach for first?

Part 05 | Transfer Learning: Why It Works

The first half of this module covered how CNNs are built. The second half covers something even more important for your career as a CV engineer: how to use a CNN that someone else has already built. This is called transfer learning, and it is the single most practical technique in modern computer vision. By the end of this part, you will understand why transfer learning works. The next part will cover how to actually use it.

The Problem With Training From Scratch

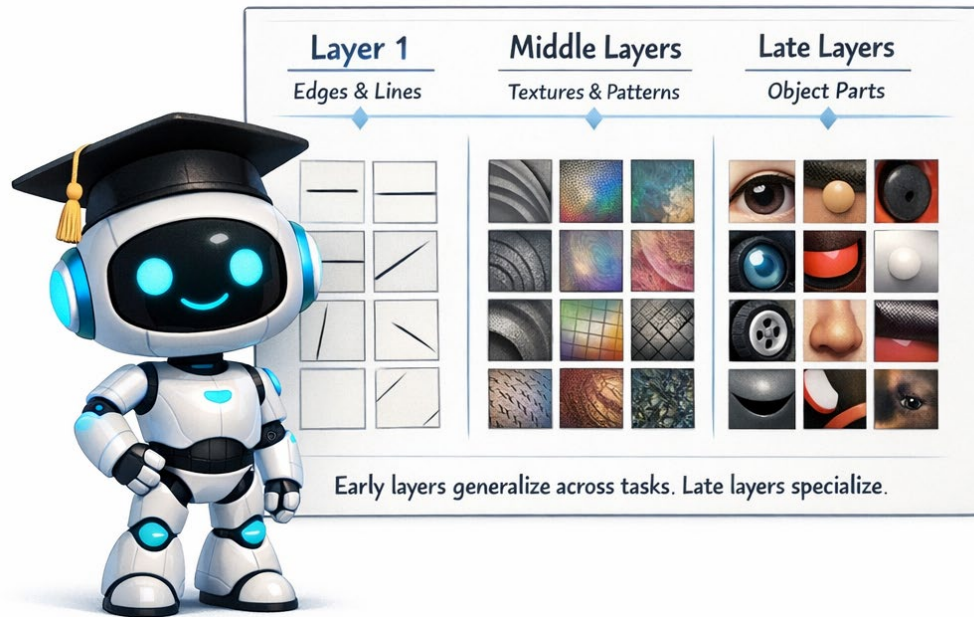
Training a CNN from scratch is expensive. AlexNet was trained on 1.2 million images for several days on a GPU that cost thousands of dollars in 2012. Modern architectures like ResNet50 take similar time on similar hardware. If you wanted to train a CNN from scratch on your own problem, you would need a comparably large labeled dataset (often you do not have one) and a lot of compute. Most CV engineers do not have either.

Even if you did have both, training from scratch is wasteful. The CNN spends most of its early training learning what edges and textures look like. But edges and textures are universal. A network that has already learned them on cat photos can recognize them just as well in chest X rays, satellite images, or factory inspection photos. Throwing away that knowledge and starting over for every new problem makes no sense. Transfer learning is the discipline of not doing that.

The Universal Visual Features

If you train a CNN on ImageNet (1.2 million natural images) and then visualize what each layer learned, a consistent pattern emerges. The first layer learns oriented edge detectors, color blob detectors, and simple textures. These look almost identical to filters that biologists have measured in animal visual cortex. They are universal. They are useful for almost any image problem, whether the images are cats and dogs or medical scans or industrial parts.

The middle layers learn more complex features: corners, curves, textures, simple shapes. These are also broadly useful but slightly more specialized. The deepest layers learn very specific features tied to the original task, like 'is this a Yorkshire terrier' or 'is this a German shepherd'. These deep features do not transfer well to other tasks because they are too specific.



The Transfer Learning Insight

Once you see the universal versus task specific gradient, the recipe writes itself. Take a network that has been pretrained on a large general dataset (ImageNet is the standard). Keep most of it, especially the early layers, because they already know what universal visual features look like. Replace just the last layer or two with new layers specific to your task. Train only those new layers, or train them and let some of the deeper original layers adjust too. Either way, you get the benefit of the pretrained knowledge without paying the full cost of training from scratch.

DEEPER LOOK | How the pretrained model gets reused

Think of the pretrained network as a feature extractor with a classifier glued on the end. The feature extractor is the convolutional layers; they take an image and produce a compact set of high level features. The classifier is the final fully connected layer; it takes those features and produces class probabilities. Transfer learning keeps the feature extractor (because edges and textures are universal) and replaces the classifier (because your task has different classes). All the math happens inside the framework; you just specify which parts to keep and which to swap.

Three Strategies on a Spectrum

Transfer learning comes in three main flavors that sit on a spectrum from most conservative to least conservative. The right choice depends on how much data you have and how similar your task is to the pretraining task.

Strategy 1: Feature Extraction (Freezing)

In feature extraction, you freeze every layer of the pretrained network. You do not change any of its weights. You add a new classifier head on top and train only that. This is the safest and fastest option. It works well when your dataset is small (a few hundred examples per class) and your task is similar to the pretraining task. Risk of overfitting is very low because almost nothing is being learned.

Strategy 2: Fine Tuning

In fine tuning, you start with the pretrained weights but allow some of the deeper layers to adjust during training along with the new classifier head. The learning rate is kept low so the original knowledge is not destroyed. This usually gives better accuracy than feature extraction, but requires more data (a few thousand examples per class is a good rule of thumb) and more careful tuning. Fine tuning is the most common choice in production.

Strategy 3: Training From Scratch

This means starting with random weights and not using any pretrained model. Reserved for cases where your domain is so different from natural images that the pretrained features would not help, like very specialized scientific imagery. You also need a very large labeled dataset (often millions of examples). In practice, this is rare; almost everyone in CV in 2026 uses some form of transfer learning.

 INDUSTRY PRACTICE | Almost nobody trains from scratch anymore

If you talk to any working CV engineer in 2026 and ask them when they last trained a model from scratch, the answer will probably be 'never' or 'years ago'. The default workflow is: pick a pretrained model, swap the head, fine tune. The cost of training from scratch is so high and the quality gap so small that it is almost never worth it. Understanding this changes how you read CV job postings: they mostly want people who can pick the right pretrained model and fine tune it well, not people who can design architectures from scratch.

Why Pretrained on ImageNet Works for Almost Everything

ImageNet is a dataset of 1.2 million natural photographs across 1000 categories: animals, objects, scenes, people. It might seem strange that a network pretrained on cat and dog photos would help with medical X rays or satellite imagery or factory defect detection. But it does, almost every time. The reason is the universal feature property: edges, textures, and shapes are useful regardless of the specific subject matter. The network has already done the hard work of learning to see; you just teach it what to look for.

There are exceptions. Some highly specialized domains (medical imaging, hyperspectral satellite data, certain scientific microscopy) benefit from pretrained models specific to their domain when those exist. But ImageNet pretraining is the safe default starting point for almost any image classification task in 2026.

 MISCONCEPTION ALERT | Bigger pretrained model is not always better

Beginners sometimes assume that the biggest pretrained model (ResNet152, EfficientNetB7, ViT Huge) will give the best results on their task. Not necessarily. For small datasets, bigger models often overfit. The right model size depends on your data size, your inference budget, and your accuracy requirements. The standard advice: start with a medium model like ResNet50 or EfficientNetB0, get a baseline, then scale up only if accuracy is the bottleneck.

 KEY TAKEAWAY

Transfer learning works because early CNN layers learn universal visual features (edges, textures, shapes) that transfer across tasks, while late layers learn task specific features that do not. Take a pretrained network, keep the early layers, replace the late layers, train on your task. Three strategies on a spectrum: feature extraction (freeze everything, train new head), fine tuning (let deeper layers adjust too), training from scratch (rare in practice). ImageNet pretrained models are the safe default.

 KNOWLEDGE CHECK | Quick check on why transfer learning works

1. Why are early CNN layers more transferable than late layers?
2. Name the three transfer learning strategies in order from most conservative to least conservative.
3. A team has 800 labeled images of healthy versus diseased leaves. Should they train from scratch, fine tune, or use feature extraction? Why?

Part 06 | Transfer Learning in Practice

This part covers the practical workflow for transfer learning: where to get pretrained models, how to set up fine tuning, what to do about small datasets, and the common pitfalls that trip up beginners. By the end of this part, you will be ready for the lab.

Where to Get Pretrained Models

Several major libraries make pretrained CV models freely available. Knowing where each lives is part of the workforce baseline.

torchvision (PyTorch)

Torchvision is the standard library for CV in PyTorch. It includes pretrained versions of ResNet, VGG, EfficientNet, DenseNet, and many others. Loading a pretrained ResNet50 is one line of code. Torchvision also includes standard transforms, datasets, and utilities, making it the easiest starting point for a new project.

Hugging Face Hub

Hugging Face has become the central hub for pretrained models across all of AI, including computer vision. It hosts thousands of pretrained vision models from various research groups and companies. The library is free, the models are usually open source, and the loading API is consistent. Hugging Face is especially strong for modern foundation models and vision transformers.

timm (PyTorch Image Models)

Timm is a curated library of over 700 pretrained vision models, all in PyTorch. It is the standard for serious CV practitioners because it includes very recent architectures, has consistent benchmarking, and is well maintained. The learning curve is slightly steeper than torchvision but the model selection is much broader.

TensorFlow Hub and Keras Applications

If you work in the TensorFlow ecosystem, TensorFlow Hub and Keras Applications are the equivalents. They cover similar architectures with similar APIs. TensorFlow is less common in cutting edge research in 2026 but still used heavily in production at many companies.

** CAREER CONNECTION | What to put on your resume**

When applying for CV jobs in 2026, you want to demonstrate hands on experience with at least one of these libraries. Torchvision is the most common requirement; almost every CV job posting mentions PyTorch. Hugging Face Transformers (or the vision counterpart) is increasingly mentioned, especially at companies working with modern foundation models. timm is a strong differentiator for research adjacent roles. Mention by name the specific architectures you have used, not just 'CNNs'.

The Fine Tuning Workflow in Four Steps

Once you have picked a pretrained model, the fine tuning workflow is straightforward.

Step 1: Load the Pretrained Model

In torchvision, this is one line: import a model and pass weights to load the pretrained version. The model now has the weights from its original ImageNet training, ready to use.

Step 2: Replace the Final Layer

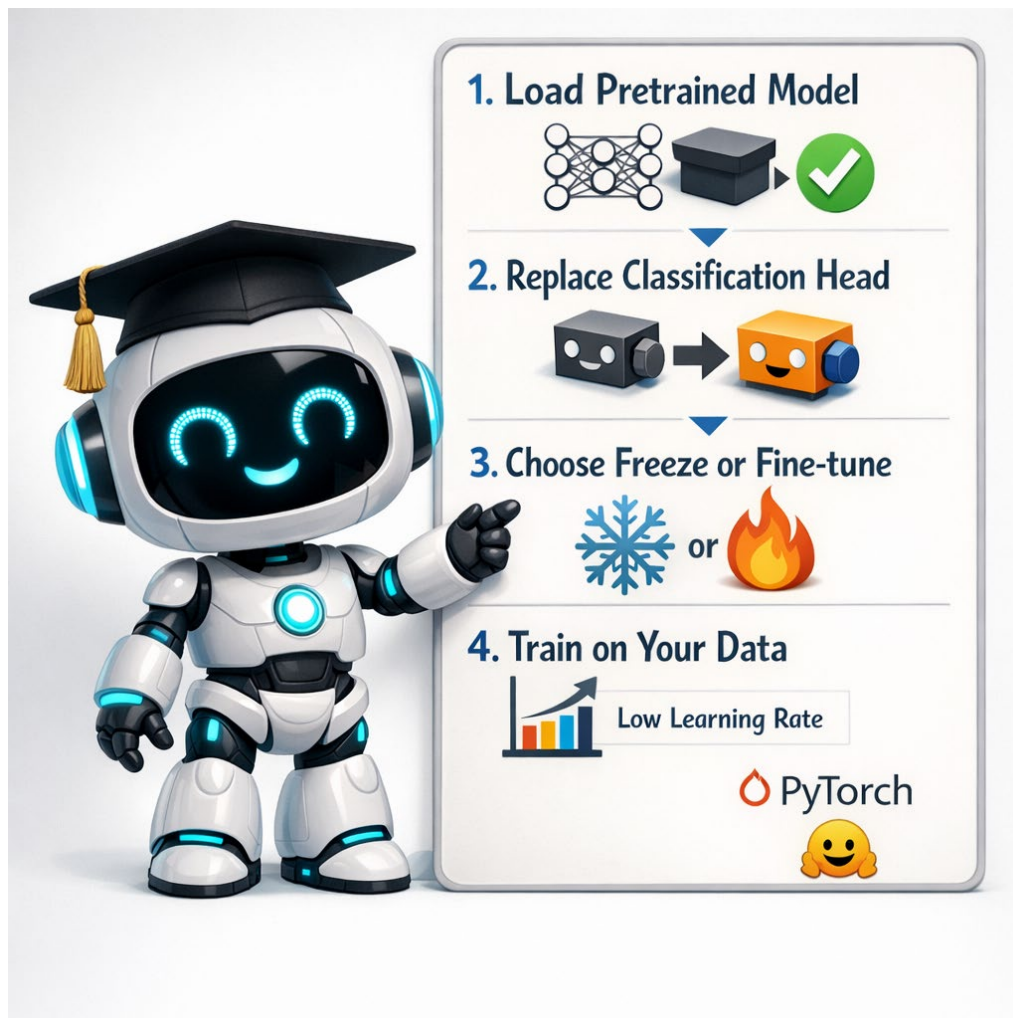
The pretrained model has a final layer designed for the original task (often 1000 classes for ImageNet). Replace it with a new layer that has the right number of outputs for your task. If you are classifying 5 plant species, that means a new layer with 5 output neurons. The new layer is randomly initialized.

Step 3: Choose What to Freeze

Decide whether to freeze the rest of the model (feature extraction) or let some of the deeper layers train along with the new head (fine tuning). For feature extraction, set the `requires_grad` flag to False on all the original parameters. For fine tuning, leave it as True but typically use a smaller learning rate.

Step 4: Train on Your Data

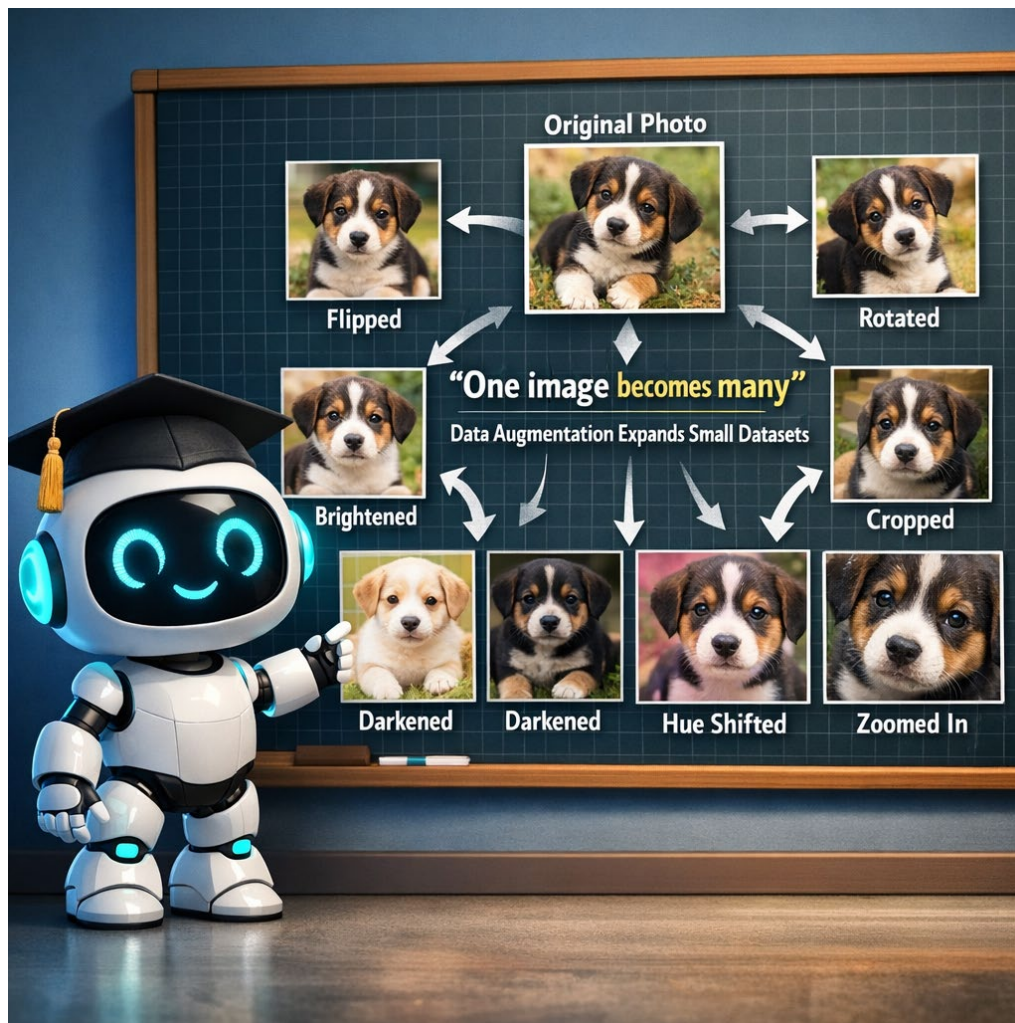
Train the modified model on your data, using standard training loops from M04. Monitor the validation accuracy. Stop when it stops improving. Save the model weights. Done.



Data Augmentation: Stretch Your Small Dataset

Even with transfer learning, you usually want more data than you have. Data augmentation creates new training examples from your existing ones by applying random transformations: horizontal flips, small rotations, random crops, brightness adjustments, hue shifts. The network sees more variety during training and generalizes better.

All major CV frameworks include built in transforms for augmentation. In PyTorch you compose them into a pipeline that runs on every training image. Some augmentations are universal (horizontal flip is safe for most natural images). Some are domain specific (horizontal flip is wrong for text or street signs that have a left right orientation). Always think about whether each augmentation makes sense for your task before applying it.



Picking the Right Strategy

Two questions decide your strategy. How much data do you have? How similar is your task to the pretraining task?

Small dataset (under 1000 examples per class) and similar domain to ImageNet (natural photos of common objects): feature extraction with the new head only. Freeze everything else. Use heavy data augmentation.

Medium dataset (1000 to 10000 per class) and similar domain: fine tune the last few blocks of the network along with the new head. Keep early layers frozen. Use moderate augmentation.

Large dataset (more than 10000 per class) and similar domain: fine tune most or all of the model. Lower learning rate than training from scratch. Light to moderate augmentation.

Different domain (medical, satellite, microscopy): try domain specific pretrained models first. If none exist, fine tune more aggressively. Consider larger augmentation rotations or domain specific augmentations.

 INDUSTRY PRACTICE | The fine tuning learning rate trick

When fine tuning, use a learning rate that is ten to a hundred times smaller than you would use for training from scratch. The pretrained weights are already good. Big steps will destroy what the network already knows. Common starting point: learning rate 0.0001 with the Adam optimizer. If the loss curve is jagged, drop the learning rate further. If it is too flat, raise it slightly. Reading the loss curve is the single most important debugging skill in deep learning.

Common Pitfalls

Three pitfalls trip up beginners more often than any others. Avoiding them puts you ahead of most newcomers.

Pitfall 1: Wrong Input Preprocessing

Every pretrained model expects a specific input format. Specific image size, specific normalization (mean and standard deviation values that the model was trained with), specific channel order. Use the wrong preprocessing and accuracy collapses to near random. Every model in torchvision and Hugging Face has documented preprocessing. Use it exactly as specified.

Pitfall 2: Wrong Learning Rate

Fine tuning needs a smaller learning rate than training from scratch. If you copy a training script that uses learning rate 0.01 (standard for from scratch) and use it on a pretrained model, the model will catastrophically forget what it learned. Start at 0.0001 or smaller for fine tuning.

Pitfall 3: Training Too Long

Small datasets overfit fast. The training loss keeps going down while the validation loss has already started going up. Monitor validation loss every epoch. Stop training as soon as it stops improving. Often two to ten epochs is enough for transfer learning. Beginners assume more training is always better; with small datasets and pretrained models, it usually is not.

 THINK ABOUT IT | Why a small learning rate matters for fine tuning

The pretrained model has spent weeks of compute learning useful features. When you fine tune, you want to gently adjust those features to your task, not throw them away. A large learning rate makes large steps, which can destroy the pretrained knowledge in the first few minibatches before any useful adjustment happens. A small learning rate respects what the model already knows and only adjusts what is needed. The phrase to remember: do not bulldoze the foundation. Renovate it carefully.

What You Can Actually Build

With everything in this module in hand, here is what you can build today as a CV engineer.

Custom image classifier for any domain you have data for. A few hundred labeled examples per class, a pretrained ResNet50 or EfficientNetB0, an afternoon of fine tuning, and you have a working system. Examples: defect inspection on a factory line, plant disease identification from leaf photos, skin lesion screening for telemedicine, product categorization for retail.

Feature extractor for downstream tasks. Use a pretrained CNN to convert images into compact feature vectors. Then use those vectors for any downstream task: clustering, retrieval, similarity search. The CNN handles the visual understanding; you handle the application logic.

Real time CV applications on consumer hardware. EfficientNetB0 on a modern laptop or even on a phone can do real time image classification at video frame rates. With transfer learning, you can adapt it to your specific use case in hours, not weeks.

 KEY TAKEAWAY

Transfer learning makes you productive in computer vision in hours, not months. Workflow: pick a pretrained model from torchvision or Hugging Face, replace the final layer, freeze or fine tune, train on your data with a small learning rate and good augmentation. Avoid the three common pitfalls: wrong preprocessing, wrong learning rate, overtraining. Match the strategy to your dataset size and domain similarity. This is the workforce baseline for CV in 2026.

☒ **KNOWLEDGE CHECK** | Quick check on practice

1. Name two libraries you would use to load a pretrained CNN, and which framework each belongs to.
2. What is the typical learning rate range for fine tuning, and why is it smaller than for training from scratch?
3. A teammate's fine tuned model gets near random accuracy. Name one likely cause and one fix.

Review Questions

Work through these on your own first. Then check your answers against the booklet and your notes. If any question feels uncertain, that is the section to reread.

1. Explain in plain English why a basic neural network (MLP) struggles to handle images. Give two distinct reasons.
2. Describe the three big architectural ideas behind convolutional neural networks. Give one sentence per idea.
3. What does a convolutional filter do? Walk through what happens at a single position when the filter is applied.
4. Define feature map and explain how it differs from the input image.
5. Compare max pooling and average pooling. Which is the default in modern CNNs and why?
6. Walk through the full CNN recipe from input image to class prediction, naming each operation in order.
7. What is the hierarchical feature learning property of CNNs? Why is it important?
8. Name five classic CNN architectures in chronological order and state the main contribution of each.
9. Why is ResNet50 considered the workhorse of production CV systems?
10. Explain in plain English why transfer learning works. What property of CNNs makes it possible?
11. Compare feature extraction and fine tuning. When should you use each?
12. Walk through the four step transfer learning workflow.
13. Why do fine tuned models need a smaller learning rate than models trained from scratch?
14. Name three common pitfalls in transfer learning and how to avoid each.
15. How does data augmentation help with small datasets, and what kinds of operations are common?

Glossary

Key terms used throughout this module, in alphabetical order.

Activation function

A function applied after every convolutional or fully connected layer that introduces non linearity. ReLU is the default in modern CNNs.

AlexNet

An 8 layer CNN that won the 2012 ImageNet competition and started the modern deep learning era in computer vision.

CNN backbone

The convolutional layers of a network, used as a feature extractor. Common backbones include ResNet50, EfficientNetB0, and VGG16.

Convolution

The core operation of a CNN. A small filter slides across the input, computing a weighted sum at each position to produce a feature map.

Data augmentation

Artificially expanding a training dataset by applying random transformations like flips, rotations, crops, and color shifts.

EfficientNet

A family of CNN architectures (2019) that scale depth, width, and resolution together according to a principled rule, achieving strong accuracy with fewer parameters.

Feature extraction

Transfer learning strategy where every layer of the pretrained network is frozen and only a new classification head is trained.

Feature map

The output of a convolutional layer. Highlights where the filter pattern was found in the input.

Filter (kernel)

A small matrix of numbers, usually 3 by 3 or 5 by 5, that serves as a pattern detector. The values are learned during training.

Fine tuning

Transfer learning strategy where the new classification head and some deeper layers of the pretrained network are trained together, usually with a low learning rate.

Fully connected layer

A layer where every input is connected to every output. Used as the final classifier on top of convolutional layers in a CNN.

Hugging Face Hub

A library and online hub hosting thousands of pretrained AI models, including CV models. The central source for foundation models.

ImageNet

A dataset of 1.2 million labeled natural images across 1000 classes. The standard pretraining dataset for transfer learning.

LeNet

A 5 layer CNN (1998) by Yann LeCun. The first practical CNN, used for reading handwritten digits.

Padding

Adding extra zero pixels around an image so that the filter can fully overlap with edge pixels. Same padding preserves output size.

Pooling

An operation that shrinks a feature map by summarizing small windows. Max pooling takes the maximum value; average pooling takes the mean.

Pretrained model

A neural network whose weights have been learned by training on a large dataset (often ImageNet). Used as the starting point for transfer learning.

ReLU

Rectified Linear Unit. The default activation function in modern CNNs. Keeps positive values and zeroes out negative ones.

ResNet

A family of very deep CNNs (2015) that use skip connections to enable training of networks with 50, 101, or more layers.

Skip connection (residual connection)

An architectural trick where the input to a block is added directly to the output, allowing gradients to flow through deep networks.

Stride

The number of pixels a filter moves between positions. Stride 1 produces a same sized output; stride 2 produces an output half as wide and half as tall.

timm

PyTorch Image Models. A curated library of 700+ pretrained vision models, widely used by professional CV practitioners.

torchvision

The standard PyTorch library for computer vision. Includes pretrained models, datasets, and image transforms.

Transfer learning

The practice of reusing a network pretrained on one task as the starting point for a different but related task.

Translation invariance

The property of recognizing a pattern regardless of where it appears in the image. CNNs have this property naturally because of shared filter weights.

VGG

A family of deep CNN architectures (2014) characterized by simplicity (only 3 by 3 convolutions) and depth (16 or 19 layers).

References and Further Reading

The following resources are excellent next steps for deepening your knowledge of convolutional networks and transfer learning. None of them are required for this course.

Foundational Books

Szeliski, Richard. Computer Vision: Algorithms and Applications, Second Edition. Springer, 2022. A comprehensive textbook covering both classic and modern CV. Available free online from the author's website at szeliski.org/Book.

Goodfellow, Ian; Bengio, Yoshua; and Courville, Aaron. Deep Learning. MIT Press, 2016. The canonical deep learning textbook. Free at deeplearningbook.org. The chapter on CNNs is the standard reference for the math underlying everything in this module.

Elgendy, Mohamed. Deep Learning for Vision Systems. Manning Publications, 2020. An accessible 480 page introduction that covers CNNs, transfer learning, GANs, and visual embeddings, all illustrated with worked CV examples. Particularly strong for this module.

Geron, Aurelien. Hands On Machine Learning with Scikit Learn, Keras, and TensorFlow, Third Edition. O'Reilly, 2022. Practical, project oriented coverage of CNNs with worked code examples.

Recent Books (2024 to 2026)

Chollet, Francois. Deep Learning with Python, Third Edition. Manning Publications, 2025. The accessible introduction by the creator of Keras, fully revised for the current era. Excellent CNN chapters.

Huang, Howard; Stevens, Eli; Antiga, Luca; and Viehmann, Thomas. Deep Learning with PyTorch, Second Edition. Manning Publications, 2025. Hands on coverage of CNNs and transfer learning in PyTorch, the framework you will use in lab.

Elgendy, Mohamed. Grokking Deep Learning for Computer Vision. Manning Publications, forthcoming (MEAP available). A modern sequel focused on CV through deep learning.

Ayyadevara, V Kishore and Reddy, Yeshwanth. Modern Computer Vision with PyTorch, Second Edition. Packt Publishing, June 2024 (ISBN 9781803231334). Comprehensive hands on coverage of modern CV including CNNs, transfer learning, vision transformers, and foundation models.

Jha, Ashish Ranjan. Mastering PyTorch, Second Edition. Packt Publishing, May 2024. Practical PyTorch from CNNs to multimodal models, including mobile and edge deployment.

Foundational Papers

LeCun, Yann; Bottou, Leon; Bengio, Yoshua; and Haffner, Patrick. Gradient Based Learning Applied to Document Recognition. Proceedings of the IEEE, 1998. The original LeNet paper.

Krizhevsky, Alex; Sutskever, Ilya; and Hinton, Geoffrey. ImageNet Classification with Deep Convolutional Neural Networks. Advances in Neural Information Processing Systems, 2012. The AlexNet paper that started the modern deep learning era.

Simonyan, Karen and Zisserman, Andrew. Very Deep Convolutional Networks for Large Scale Image Recognition. International Conference on Learning Representations, 2015. The VGG paper.

He, Kaiming; Zhang, Xiangyu; Ren, Shaoqing; and Sun, Jian. Deep Residual Learning for Image Recognition. IEEE Conference on Computer Vision and Pattern Recognition, 2016. The ResNet paper.

Tan, Mingxing and Le, Quoc. EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. International Conference on Machine Learning, 2019. The EfficientNet paper.

Documentation and Tutorials

PyTorch torchvision documentation at pytorch.org/vision. The standard library for pretrained CV models in PyTorch. Includes worked tutorials for transfer learning.

Hugging Face Transformers documentation at huggingface.co/docs/transformers. Increasingly important for modern vision models and foundation models.

timm library documentation at huggingface.co/docs/timm. Curated PyTorch Image Models library with 700+ pretrained vision models. The professional CV standard in 2026.

Stanford CS231n: Convolutional Neural Networks for Visual Recognition. Free lectures on YouTube. Slightly older now but still the standard introduction to CNNs.

Looking Ahead

Module 06 takes the CNN foundation you just built and applies it to two harder tasks: object detection (finding where each object is in the image) and image segmentation (drawing pixel level outlines of every object). The CNN backbone and transfer learning workflow you mastered in this module become the foundation for both.

In M06 you will see how detection adds bounding boxes and class labels, learn about the YOLO family and modern transformer based detectors like RT-DETR, and meet GroundingDINO for open vocabulary detection. You will then move to segmentation: semantic, instance, and panoptic types. You will learn about U-Net and Mask R-CNN, and meet Segment Anything (SAM and SAM 2), the foundation model that changed segmentation from a custom training task into a callable service.

The midterm project (Version 1) is introduced in M06. You will design and implement a baseline of your chosen CV application using techniques from M01 through M06. Half the course done; half to go.

email or message me if you have any questions